



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

13 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 9 Lecture 1

LINKING

PART 1



Lecture Outline

- ★ **Linking**
- ★ **Static Linking**
- ★ **Dynamic Linking**
- ★ **Case study: Library inter-positioning**

LINKING

- ★ Linking is the process of collecting and combining various pieces of code and data into a single file that can be loaded (copied) into memory and executed.
- ★ Linking can be performed at compile time, when the source code is translated into machine code; at load time, when the program is loaded into memory and executed by the loader; and even at run time, by application programs.
- ★ On modern systems, linking is performed automatically by programs called **linkers**.

Example C Program

- The example program consists of two source files, main.c and sum.c.
- The main function initializes a two-element array of ints, and then calls the sum function to sum the array elements

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

main.c

```
int sum(int *a, int n)
{
    int i, s = 0;

    for (i = 0; i < n; i++) {
        s += a[i];
    }
    return s;
}
```

sum.c

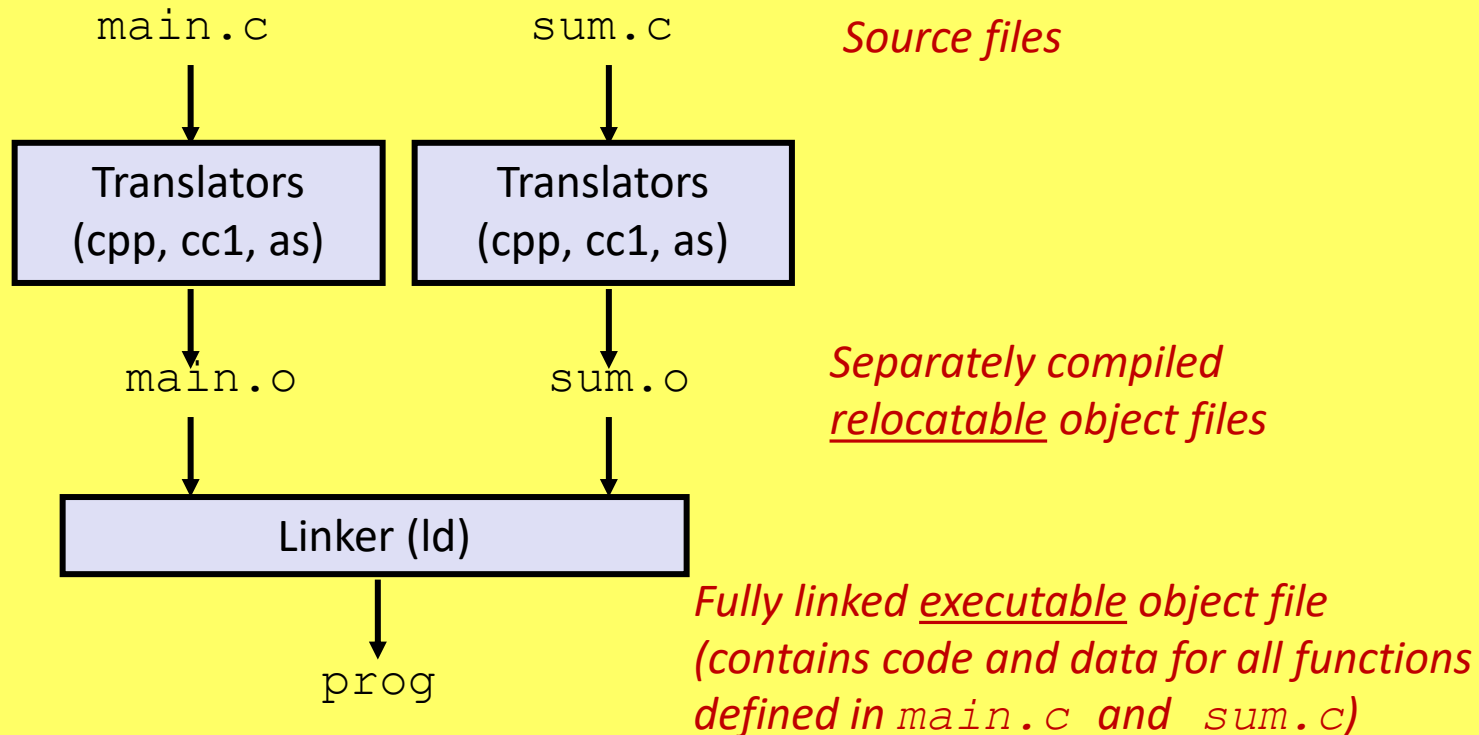
COMPILER DRIVER

Most compilation systems provide a compiler driver that invokes the language preprocessor, compiler, assembler, and linker, as needed on behalf of the user

- * Programs are translated and linked using a *compiler driver*:

```
$ gcc -Og -o prog main.c sum.c
```

```
$ ./prog
```



COMPILER DRIVER

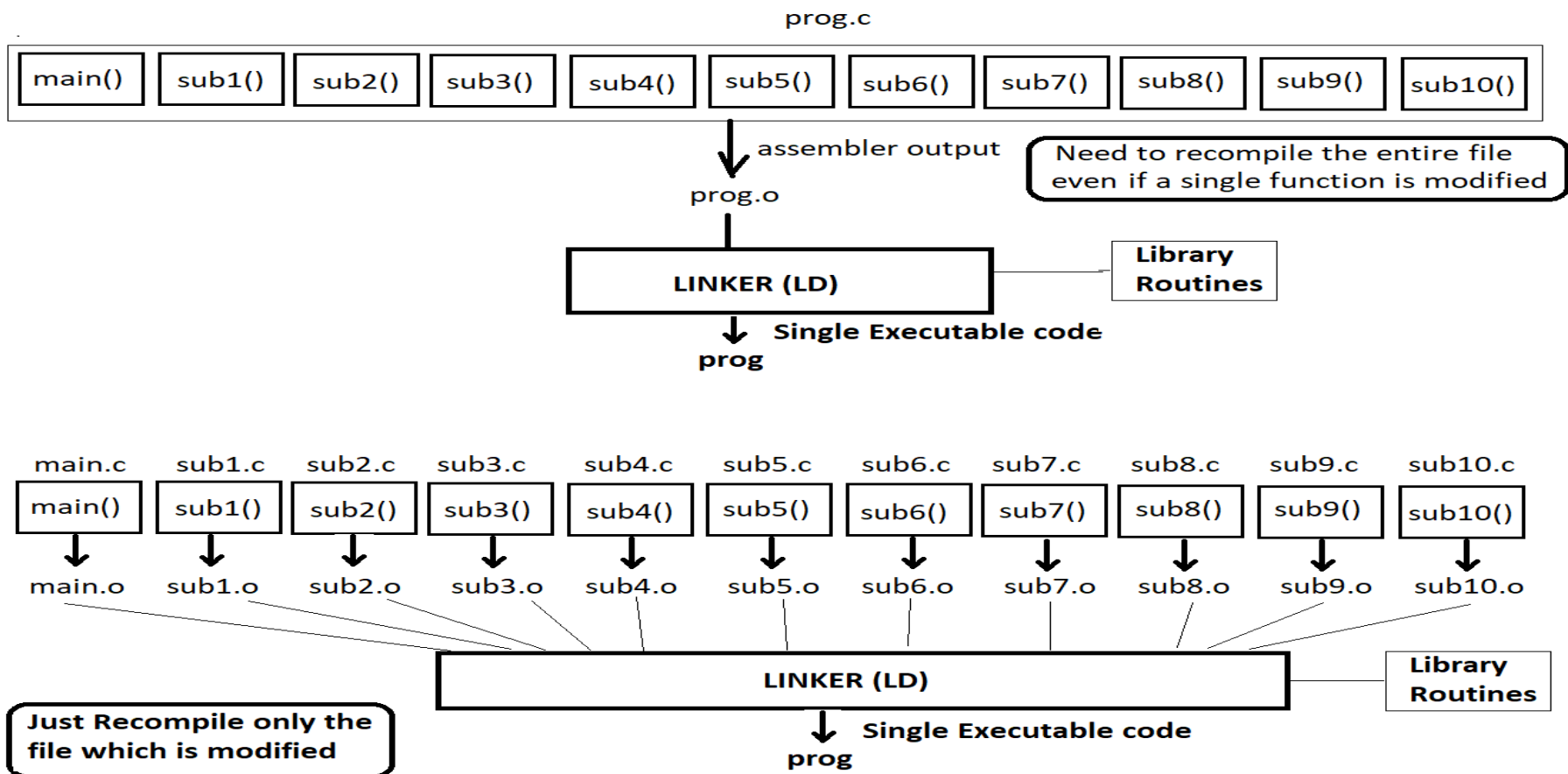
❏ Compiler Driver Steps

- * To see the Compiler Driver steps, run gcc with the -v option `gcc -v -Og -o p main.c sum.c`
- * The driver first runs the C preprocessor(cpp), which translates the C source file main.c into an ASCII intermediate file main.i:
`cpp [other arguments] main.c /tmp/main.i`
- * Next, the driver runs the C compiler(cc1), which translates main.i into an ASCII assembly language file main.s.
`cc1 /tmp/main.i main.c -O2 [other arguments] -o /tmp/main.s`
- * Then, the driver runs the assembler(as), which translates main.s into a relocatable object file main.o:
`as [other arguments] -o /tmp/main.o /tmp/main.s`
- * The driver goes through the same process to generate **sum.o from sum.c**
- * Finally, it runs the linker program ld, which combines main.o and sum.o, along with the necessary system object files, to create the executable object file p:
`ld -o p [system object files and args] /tmp/main.o /tmp/sum.o`
- * To run the executable p, we type its name on the Unix shell's command line:
`$ ./p`
- * The shell invokes a function in the operating system called the **loader**, which copies the code and data in the executable file p into memory, and then transfers control to the beginning of the program.

Why Linkers?

Reason 1: Modularity

- Program can be written as a collection of smaller source files, rather than one monolithic mass.



Why Linkers? (cont)

❑ Reason 2: Efficiency

– Time: Separate compilation

- ★ Change one source file, compile, and then relink.
- ★ No need to recompile other source files.

– Space: Libraries

- ★ Common functions can be aggregated into a single file...
- ★ Can build libraries of common functions
e.g., Math library, standard C library
- ★ Yet executable files and running memory images contain only code for the functions they actually use.

LINKING TYPES

❑ TWO KINDS OF LINKING

❖ **STATIC LINKING**

- ★ Linking performed at compile time, when the source code is translated into machine code;

❖ **DYNAMIC LINKING**

- ★ Linking performed at load time, when the program is loaded into memory
- ★ Linking performed at **run time**

Static Linking

- ★ Static linkers such as the Unix ld program take as input a collection of relocatable object files and command-line arguments and generate as output a fully linked executable object file that can be loaded and run.
- ★ The input relocatable object files consist of various code and data sections.
- ★ Instructions are in one section, initialized global variables are in another section, and uninitialized variables are in yet another section

Linker Tasks

- ★ To build the executable, the linker must perform two main tasks:

STEP 1 : Symbol resolution.

- ★ Object files define and reference symbols. The purpose of symbol resolution is to associate each symbol reference with exactly one symbol definition.

STEP 2 : Relocation.

- ★ Compilers and assemblers generate code and data sections that start at address 0.
- ★ The linker relocates these sections by associating a memory location with each symbol definition, and then modifying all of the references to those symbols so that they point to this memory location.
- ★ The linker blindly performs these relocations using detailed instructions, generated by the assembler called relocation entries.

What Do Linkers Do?

□ Step 1: Symbol resolution

- Programs define and reference *symbols* (global variables and functions):

```
void swap() {...}    /* define symbol swap */
swap();              /* reference symbol swap */
int x=100;            /* here x is defined */
int y = x;           /* here x is referenced and y is
                      defined */
int *xp = &x; /* define symbol xp, reference x */
```

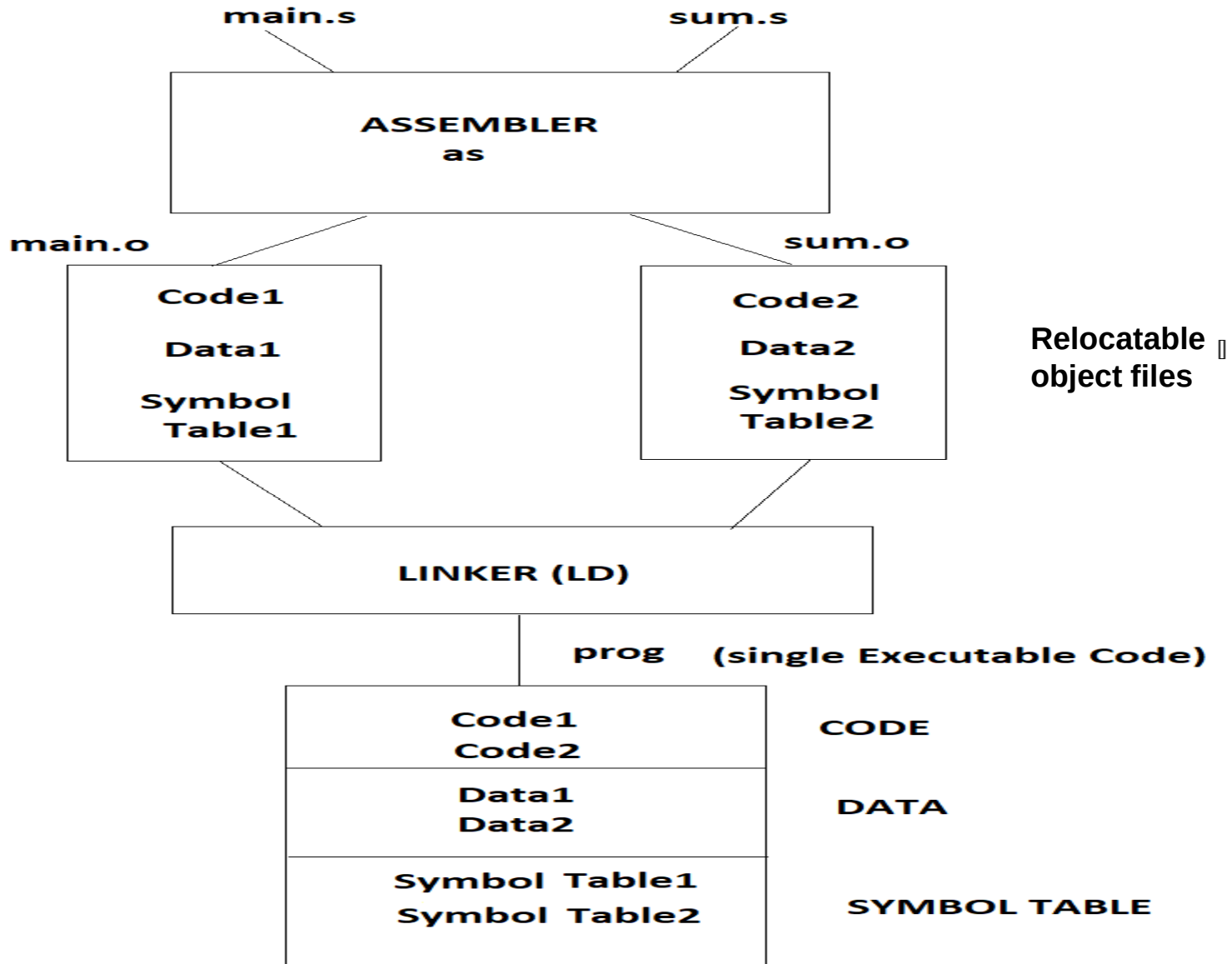
- Symbol definitions are stored in object file (by assembler) in *symbol table*.
 - * Symbol table is an array of structs
 - * Each entry includes name, size, and location of symbol.
- **During symbol resolution step, the linker associates each symbol reference with exactly one symbol definition.**

What Do Linkers Do? (cont)

□ Step 2: Relocation

- Merges separate code and data sections into single sections
- Relocates symbols from their relative locations in the `.o` files to their final absolute memory locations in the executable.
- Updates all references to these symbols to reflect their new positions.

What Do Linkers Do?



Three Kinds of Object Files (Modules)

* Relocatable object file (.o file)

- Contains code and data in a form that can be combined with other relocatable object files to form executable object file.
 - * Each .o file is produced from exactly one source (.c) file

* Executable object file (a.out file)

- Contains code and data in a form that can be copied directly into memory and then executed.

\$ gcc prog.c generates by default the executable file **a.out**

We change the name of the executable file by using the option -o in gcc command \$gcc -o prog prog.c which generates the executable file with the name prog instead of a.out

* Shared object file (.so file)

- Special type of relocatable object file that can be loaded into memory and linked dynamically, at either load time or run-time.
- **Called *Dynamic Link Libraries* (DLLs) by Windows**

Executable and Linkable Format (ELF)

- ★ Object file formats vary from system to system.
- ★ The first Unix systems from Bell Labs used the **a.out** format.
- ★ Early versions of System V Unix used the **Common Object File format (COFF)**.
- ★ Windows NT uses a variant of COFF called the **Portable Executable (PE) format**.
- ★ Modern Unix systems—such as Linux, later versions of System V Unix, BSD Unix variants, and SunSolaris—use the Unix **Executable and Linkable Format (ELF)**
- ★ Standard binary format for object files
- ★ One unified format for
 - Relocatable object files (`.o`),
 - Executable object files (`a.out`)
 - Shared object files (`.so`)
- ★ Generic name: ELF binaries

ELF Object File Format

A typical ELF relocatable object file contains the following :

Elf header

begins with a 16-byte sequence that describes the **word size and byte ordering of the system that generated the file.**

The rest of the ELF header contains information that allows a linker to parse and interpret the object file.

This includes the **size of the ELF header, the object file type (e.g., relocatable, executable, or shared), the machine type (e.g., IA32), the file offset of the section header table, and the size and number of entries in the section header table.**

Segment header table

Page size, virtual addresses of memory segments (sections), segment sizes.

.text section

Code – machine code of the compiled program

.rodata section

Read only data such as format strings in printf statements & jump tables for switch statements

.data section

Initialized global and static C variables. Local C variables are maintained at run time on the stack and do not appear in either the .data or .bss sections

.bss section

Uninitialized global and static C variables

“Block Started by Symbol”(IBM 704 As lang), **“Better Save Space”**, Has section header but occupies no space

ELF header
Segment header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
.line section
.strtab section
Section header table

ELF Object File Format (cont.)

.symtab section

Symbol table with information about functions and global variables that are defined and referenced in the program

Procedure and static variable names

Section names and locations

Every relocatable object file has a symbol Table in .symtab

.rel.text section

Relocation info for .text section

A list of locations in the .text section that will need to be modified when the linker combines this object file with others. In general, any instruction that calls an external function or references a global variable will need to be modified.

.rel.data section

Relocation info for .data section, Addresses of pointer data that will need to be modified in the merged executable. Relocation information for any global variables that are referenced or defined by the module

.debug section

Info for symbolic debugging (gcc -g), A debugging symbol table with entries for local variables and typedefs defined in the program, global variables defined and referenced in the program, and the original C source file. It is only present if the compiler driver is invoked with the -g option.

.line section

Mapping between line numbers in the original C source program and machine code instructions in the .text section

.strtab section

A string table for the symbol tables in the .symtab and .debug sections and for section names in the section headers

Section header table

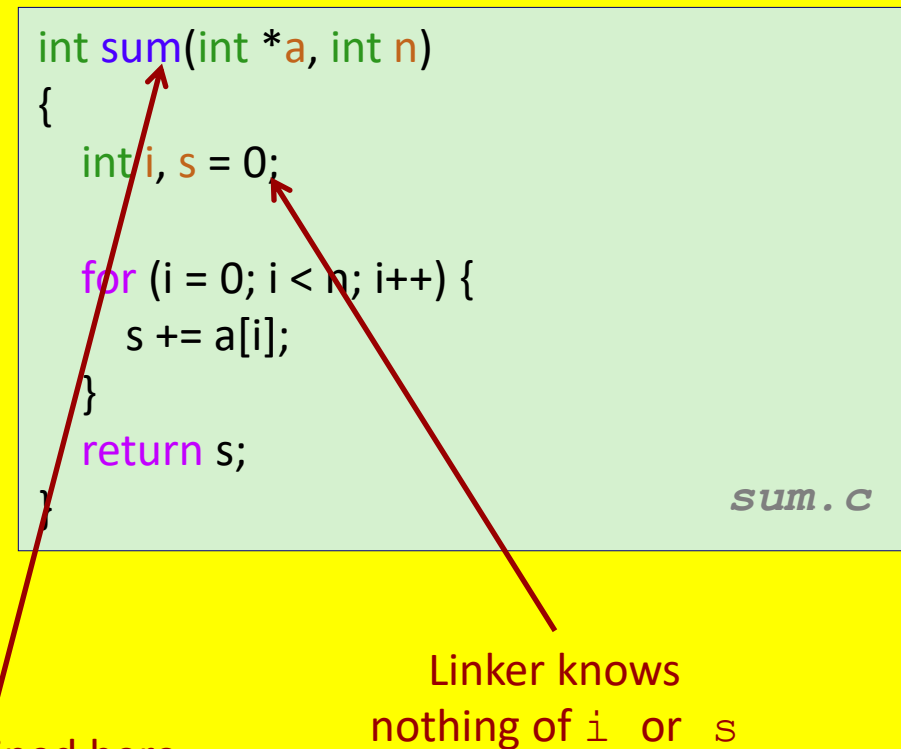
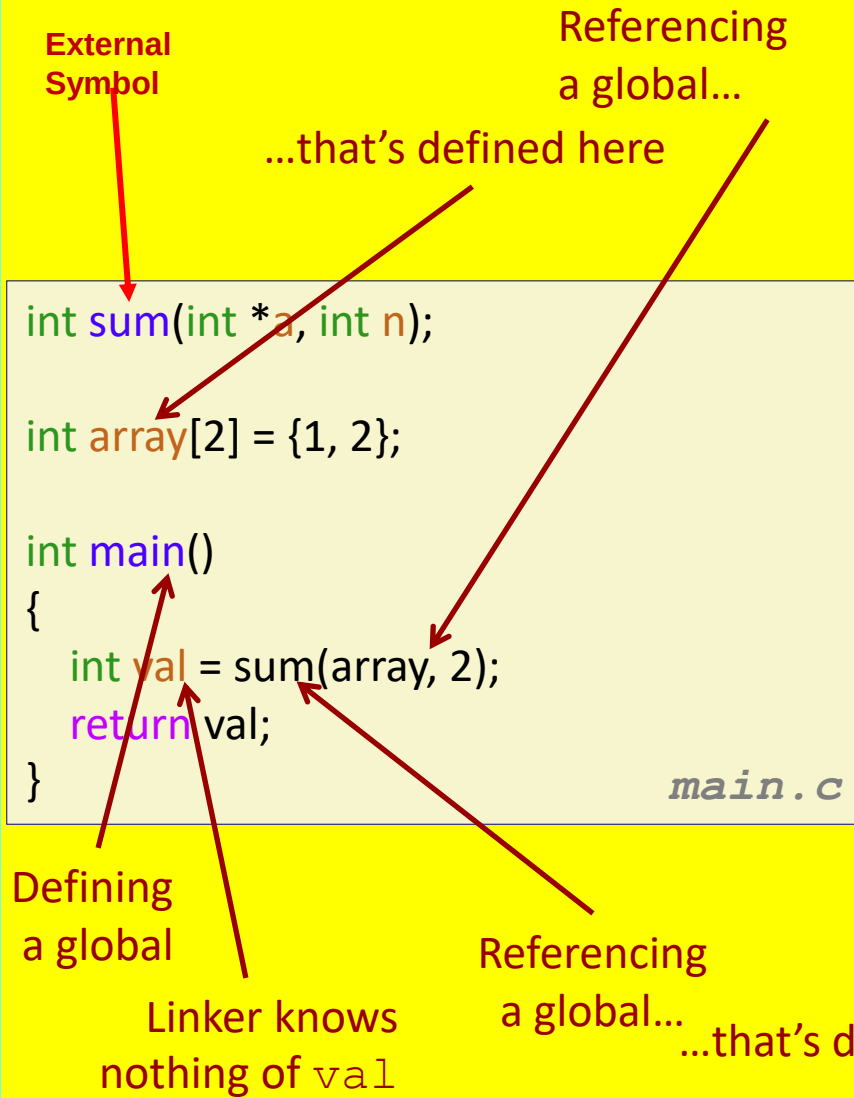
Offsets and sizes of each section

ELF header
Segment header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
.line section
.strtab section
Section header table

Linker Symbols

- ★ Each relocatable object module m has a symbol table that contains information about the symbols that are defined and referenced by m . In the context of a linker, there are 3 different kinds of symbols
- ★ **Global symbols**
 - Symbols defined by module m that can be referenced by other modules.
 - E.g.: non-**static** C functions and non-**static** global variables.
- ★ **External symbols**
 - Global symbols that are referenced by module m but defined by some other module.
- ★ **Local symbols**
 - Symbols that are defined and referenced exclusively by module m .
 - E.g.: C functions and global variables defined with the **static** attribute.
- **Local linker symbols are *not* local program variables**

Step 1: Symbol Resolution



Local Symbols

- ★ Local non-static C variables vs. local static C variables
 - local non-static C variables: stored on the stack
 - local static C variables: stored in either `.bss`, or `.data`

```
int f()
{
    static int x = 0;
    return x;
}

int g()
{
    static int x = 1;
    return x;
}
```

Compiler allocates space in `.data` for each definition of `x`

Creates local symbols in the symbol table with unique names, e.g., `x.1` and `x.2`.



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

13 April 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG